Incremental Hybrid RAG Architecture Proposal

Date: 2026-10-14

Status: **Design for review only. Nothing in this proposal is implemented.**

Approved scope for this turn: design only. Proposed rollout: opt-in, with current retrieval retained by default. No source files, environment variables, database objects, UI, or running services were changed to create this document.

1. Recommendation

Evolve the existing application into **evidence-grounded hybrid retrieval RAG**:

Authorized scope -> dense + lexical retrieval -> rank fusion -> bounded context selection -> existing grounded generation -> validated evidence.

Keep FastAPI, React, the existing ingestion modules, local embeddings, FAISS/pgvector choices, and the monolith/optional service-split topology. Add functionality inside those boundaries rather than replacing the project.

This is still a pipeline, not an autonomous agent. Bounded agentic retrieval is a later optional capability, not a prerequisite for hybrid search.

There is no universally "best" architecture without a representative workload. The recommendation prioritizes correctness, source traceability, reversibility, and measured improvements over adding frameworks or infrastructure.

Reciprocal-rank fusion combines the two ranked lists without treating their incompatible scores as equivalent. It can run within your existing FAISS/pgvector setup—no replacement application, new UI, or autonomous agent is needed.

2. Existing implementation versus proposed capabilities

Area	Verified existing behavior	Proposed evolution
Ingestion	`app\main.py` indexes extracted text through shared chunk/embed/store helpers.	Retain the flow; improve source identity, replacement atomicity, and provenance incrementally.
Embeddings	Local `all-MiniLM-L6-v2`; dense query/chunk vectors.	Retain model initially; version embeddings and evaluate alternatives later.
Retrieval	Vector search followed by lexical reranking of the vector candidate pool.	Independently retrieve lexical candidates as well, then fuse both ranked lists.
Storage hybrid	`HybridVectorStore` mirrors FAISS/PG writes and searches one primary.	Preserve this meaning; do not confuse it with lexical+dense retrieval.
Ranking	Heuristic query-term overlap.	Keep dense-only behavior unchanged; use fusion as hybrid primary order with deterministic lexical tie-breaking.

Area	Verified existing behavior	Proposed evolution
Generation	OpenAI/Hugging Face/auto or optional inference service.	Retain integrations; explicitly bounded timeouts and validated output are later improvements.
Evidence	Text section labels and prompt-local CTX identifiers.	Preserve current answer text; later add validated structured source identities.
Scope	Client-supplied equality filters, not server authorization.	Fix isolation first; introduce authenticated server-derived scope before wider exposure.
Evaluation	Unit tests and synthetic metrics dashboard.	Add a labeled non-sensitive benchmark and actual retrieval/evidence metrics.
Agents	No autonomous planning/retrieval loop.	Consider a bounded read-only retry loop only after baseline evaluation.

Relevant current code:

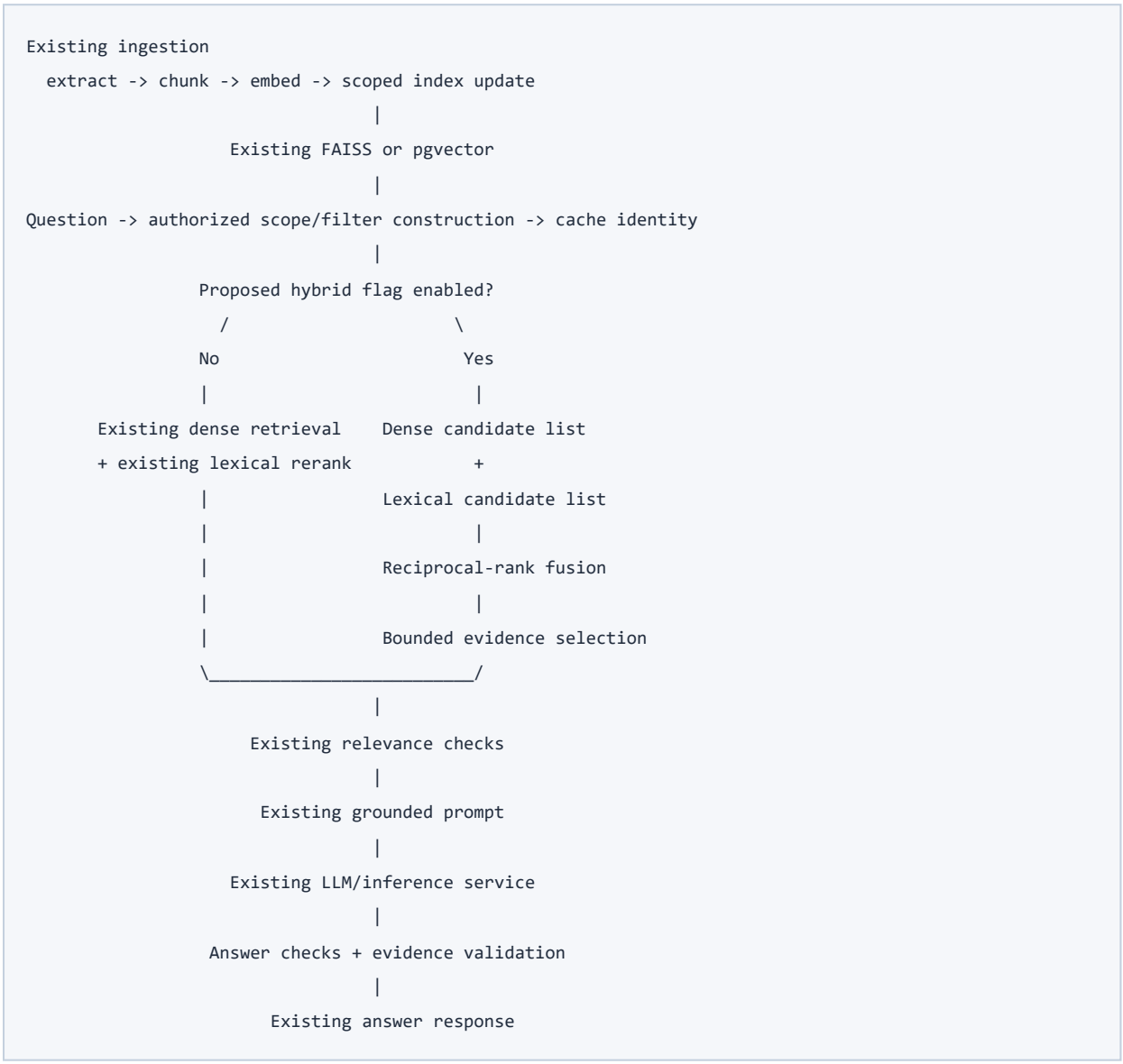
- Filters and candidate retrieval: `app\rag.py:562-698`.
- Lexical reranker: `app\rag.py:489-513`.
- Result contract/wrapper: `app\vector_store.py:29-75,467-588`.
- FAISS implementation: `app\vector_store.py:111-260`.
- PG search: `app\vector_store.py:355-387`.
- Mirrored storage backend: `app\vector_store.py:419-464`.
- Split retrieval route: `app\retrieval_service.py:31-104`.
- Public response model: `app\schemas.py:36-46`.

3. Architectural invariants

1. Preserve public endpoint paths, the current React layout, and supported deployment choices. Do not introduce a new UI or replacement service.
2. Keep direct uploads, Google Docs, SharePoint, and SQL row ingestion feeding the same pipeline. Search changes must cover API and in-process demo use.
3. The proposed feature flag defaults off. The dense-only route remains the behavioral baseline; the existing `VECTOR_DB_BACKEND=hybrid` keeps its current mirrored-storage meaning.
4. Apply the same authorized scope and metadata constraints to both retrieval branches before selecting/fusing candidates. Never retry without filters.
5. Preserve genuine source identity through retrieval, ranking, generation, and references. Do not identify evidence by text alone or positional rank.
6. Do not infer confidence from raw distance, BM25 score, FTS rank, or RRF score.
7. Keep bounded requests, candidate pools, prompt budgets, and optional agent actions. Failure must be explicit, not a success-shaped empty fallback.
8. Do not enable hybrid mode on an unsupported/mismatched backend or topology. Configuration/readiness should fail clearly, not silently select dense mode.

9. Add no external search service, graph database, agent framework, Redis, or local LLM in the initial phase. Justify later dependencies with evidence.
10. Do not promise byte-for-byte preservation of known erroneous behavior: prerequisite bug fixes should intentionally remove cache/source collisions while preserving correct outputs and API contracts.

4. Target retrieval flow



The initial hybrid implementation should change retrieval, not provider selection or autonomous planning. Stronger structured evidence validation is a separately reviewed phase, not something claimed to exist today.

5. Retrieval design

5.1 Dense branch

Use the existing local embedding model and selected backend. Fetch a bounded candidate pool under the complete filter set.

Keep L2 semantics explicit: FAISS currently returns squared L2; pgvector `<->` returns L2. Rank fusion uses ranks, so these numerical scales do not need normalization. Real distances are diagnostics, not fused confidence.

5.2 Lexical branch

The lexical branch must search the authorized corpus independently, not merely rescore the dense candidates. Otherwise terms missed by dense retrieval can never enter the final context.

Recommended backend-specific starting point:

Backend	Proposed lexical retrieval	Rationale/limitations
FAISS	Small in-process BM25 index over the existing chunk records, kept consistent on add/clear/delete.	Exact-term recovery without new infrastructure. Scope-restricted scoring/statistics avoid ranking influenced by inaccessible documents. Benchmark memory/update cost before scaling.
PostgreSQL/pgvector	Native PostgreSQL full-text retrieval using a matching `to_tsvector` expression/GIN index, parameterized query and identical metadata predicates.	Uses the existing durable data store and all historical chunk text; no separate search service. Native `ts_rank`/`ts_rank_cd` is **not BM25**.
Mirrored hybrid store	Run dense and lexical search on the configured primary store.	Avoid duplicate evidence and drift from combining unaligned FAISS/PG row identities.

A database-native lexical ranker is an intentional refinement of the initial BM25 suggestion: it keeps the existing PostgreSQL architecture simple. If evaluation demonstrates a need for actual BM25 on PostgreSQL, separately review an implementation/dependency; do not describe native full-text rank as BM25.

For a first PostgreSQL experiment, consider the `simple` search configuration for identifiers and multilingual/code-like tokens. Evaluate `english` stemming for prose separately. The query and indexed expression must use the same configuration. Explicitly test punctuation, product codes, numbers, acronyms, hyphens, accented text, and non-English scripts.

Use a vetted, tested BM25 implementation if a dependency is needed; do not add an unexplained custom scoring approximation. Dependency choice remains a review decision, not an installed package in this proposal.

5.3 Rank fusion

Use reciprocal-rank fusion (RRF), with 1-based positions:

```
RRF(candidate) = sum(1 / (k + rank_in_branch))
```

Initial experiment: equal branch weights and `k = 60`. This is a conventional starting value, not a claim that it is optimal for these documents.

Merge by genuine chunk identity within its index namespace and scope, not text. Keep branch ranks/scores as separate diagnostics. Define stable tie-breaking: fusion score, existing lexical relevance tie-break, then stable chunk identity. Missing branch membership contributes zero.

Start with the existing `RETRIEVAL_RERANK_POOL_SIZE` bound per branch. The current code default is 15; therefore the union is at most 30 before fusion selection. Cap the fused working pool back to the configured candidate budget, and keep the existing `TOP_K=5` final context count initially.

Do **not** blindly run the current lexical reranker as a new primary sort after RRF: it can undo the dense/lexical balance. Preserve it unchanged for dense-only mode and reuse its lexical signal only as a bounded hybrid tie-break initially. An optional learned reranker can later replace hybrid final ordering only after evaluation and a separate flag.

5.4 Empty results and failures

- A successful lexical search with zero matches is normal; use available dense candidates. A successful dense search with zero matches can use lexical ones.
- Both searches successfully empty: return the existing abstention through the normal answer path.
- Backend failure is not equivalent to zero results. Log a sanitized structured failure and surface the retrieval error under the approved API error contract.
- Do not silently downgrade to dense mode on missing lexical support, malformed request shapes, invalid scope, or a stale/unready lexical index.
- An explicitly approved degraded mode could be added later, but must disclose the mode in diagnostics and be covered by tests.

5.5 Existing relevance and grounding checks

Retain the current gates during the initial experiment. Dense/lexical fusion can increase candidate recall yet still fail the existing ASCII lexical gates on synonyms or multilingual questions. Measure:

1. Retrieval before the gates.
2. Context retained after selection.
3. Final answer/abstention after the gates.

Do not weaken grounding just to make hybrid results look better. Review a semantic relevance/claim verifier separately if evaluation shows consistent false abstentions. Synthesis/external-prefix bypass weaknesses described in `RAG_CODEBASE_ANALYSIS.md` remain prerequisites for reliable evidence claims.

6. Data contracts, indexing, and cache consistency

6.1 Genuine candidate identity

Complete the existing result refactor before fusion:

- PostgreSQL results must include the real row ID, text, metadata, source document, and distance rather than positional IDs/zero scores.
- FAISS IDs must be immutable and unique after scoped replacement/deletion.
- Use a compound evidence identity: index namespace + source version + chunk ID. A metadata-only date/tag does not uniquely identify the chunk.
- Retain separate `dense_distance`, `lexical_score`, `fusion_score`, and branch ranks internally. Do not overload one `score` with incompatible meanings.
- Two documents with identical paragraphs remain distinct evidence records. Prompt deduplication may consolidate display text only if all provenance is retained.

Prefer extending/reusing `VectorSearchResult` and a small typed ranking record over introducing a broad plugin/orchestration framework.

6.2 Ingestion consistency

FAISS lexical indexing must use the same text/metadata/IDs as its dense index. Atomic publication or a guarded revision must prevent searches from seeing mixed generations. Add/clear/delete and failed updates require regression tests.

For PostgreSQL, lexical retrieval over existing text can use an optional expression index rather than re-embedding all documents or adding a second corpus. The lexical query predicates must match dense predicates. A scoped replacement and index-generation increment should occur in one transaction.

A proposed PostgreSQL migration should:

- Validate the configured table identifier using existing helpers.
- Add only the needed lexical index and measured filter indexes.
- Preserve existing IDs, text, embeddings, metadata, and timestamps.
- Document index-build locking/cost; use concurrent index creation where operationally appropriate and outside an incompatible transaction.
- Be explicitly applied/reviewed, not triggered by an unapproved startup change.
- Leave the dense path usable if the optional lexical index is not deployed.

No migration or database change was applied for this design.

6.3 Cache identity and invalidation

Before enabling hybrid retrieval, fix the confirmed cache defects. Both answer and retrieval keys must contain:

- Store/index namespace and visible corpus generation.
- Complete normalized authorization and metadata filters.
- Dense/hybrid mode and ranking/lexical configuration version.
- Embedding model/version and relevant candidate limits.
- Prompt/provider version where it can change generated answers.
- Query identity: hybrid lexical results depend on the normalized query text, not just a rounded query embedding.

Cache raw scoped candidate lists separately from query-specific ranking where appropriate. A matching embedding alone is insufficient for lexical results.

For FAISS, an in-process generation can work for a single process. For multi-worker/split PostgreSQL operation, use a durable index-generation record in the existing database, updated with scoped writes and consulted before cache use. A small generation table is an optional schema addition to design/review, not a new infrastructure service.

If shared invalidation is not implemented, explicitly restrict the first release to the supported single-process topology. Do not advertise split or multi-worker correctness on process-local revision counters. This restriction must be checked before opt-in deployment.

6.4 Public and internal API compatibility

Public `/upload`, `/ask`, and ingestion route paths remain unchanged. `QuestionRequest` does not need a client-selectable retrieval mode initially; operators control the feature to prevent unexpected cost/scope behavior.

The existing `/search` internal service must negotiate a retrieval mode/capability and preserve structured candidates in an additive response alongside legacy `context_chunks`. The API/service versions must agree when hybrid is enabled; older service payloads must not silently lose scope or evidence.

The monolith and retrieval service should call one shared ranking helper. Duplicating fusion/filter logic across them is not acceptable. If a small `app\retrieval.py` module becomes necessary for shared pure retrieval logic, add it only at implementation time with a clearly bounded purpose.

Answer text and `question_domain` remain compatible. Structured citations can be an additive later response field; no React layout change is required to continue rendering the existing text.

7. Proposed configuration and rollout

The following flag does not exist in the current code.

```
HYBRID_RETRIEVAL_ENABLED=false
```

Proposed behavior:

Setting	Behavior
Absent/false	Current dense-only retrieval/reranking, with reviewed correctness fixes.
True + supported/ready backend	Dense plus lexical candidate retrieval and fusion.
Invalid value or incompatible/unready enabled backend/topology	Explicit validation/readiness error; no silent success-shaped fallback.

Keep RRF parameters as documented experiment constants initially rather than adding numerous environment knobs. Promote only values with demonstrated tuning need. Existing backend selection stays independent of retrieval mode.

Rollout sequence:

1. Offline benchmark on a copied/non-sensitive representative corpus.
2. Existing deterministic tests with the flag off.
3. New deterministic and backend integration tests with the flag on.
4. Enable for a trusted local environment; compare outputs on fixed questions.
5. Enable on an approved persistent deployment only after isolation and cross-worker consistency gates pass.
6. Default remains off until a separate adoption decision.

Rollback: disable the proposed flag and restart the affected API/retrieval processes, because configuration is read at import. Mode-aware keys prevent old hybrid entries contaminating dense caches. Optional indexes can remain inert; rollback should not delete corpus data or require re-embedding.

8. Phased roadmap

Phase 0: correctness and safe foundations

Fix cache filter/store isolation, stable IDs, vector result fidelity, unsafe upload staging, queued-upload client readiness, and atomic replacement. Remove unfiltered compatibility retries when scope is required.

Before wider access, add server authentication/authorization, connector allowlists/least privilege, job/feedback scope, resource caps, and build-secret exclusions. These are not automatically solved by hybrid retrieval.

Review the dirty vector-store refactor before editing it; preserve unrelated work. These bug/security fixes can be separate small commits with regression tests rather than bundled into a large retrieval redesign.

Phase 1: opt-in hybrid retrieval

Implement independent lexical retrieval, bounded RRF, common scope logic, consistent result identities, mode-aware caches, and local/service parity. Start without a new embedding model, LLM, agent framework, or UI.

Phase 2: trustworthy citations and extraction

Preserve source IDs/URLs, document versions, PDF pages, DOCX paragraph/table locations, SQL primary keys/row identifiers, and chunk offsets during ingestion. Map prompt-local CTX IDs to persistent evidence and validate emitted citations.

Add an optional structured source list without removing answer text. Do not claim fact-level support merely because an answer repeats retrieved terms. Extend extraction format by format with fixtures; do not reindex the whole corpus without a migration/version plan.

Phase 3: measured retrieval and evaluation

Replace synthetic dashboard data with versioned real evaluation artifacts. Evaluate chunk boundaries, lexical configurations, embeddings, and optional cross-encoder reranking using the same test questions/scope.

Use frozen baseline results and ablations: dense-only, lexical-only, dense+lexical+RRF, and fusion plus optional learned reranking. Evaluate full answer behavior, not just retrieval scores.

Phase 4: production reliability and performance

Narrow locks, offload synchronous work from async event loops, remove duplicate remote query embeddings, avoid embedding prewarm in generation-only services, and add readiness/structured metrics.

Share corpus generations and job state for supported multi-worker deployment. Start with the existing database where it fits; adopt a durable worker/broker only when reliability/backlog requirements justify it.

Track provider failure versus extractive fallback explicitly. Bound provider latency/cost and verify actual auto-mode behavior. These changes do not require a new frontend design.

Phase 5: optional bounded agentic retrieval

Consider only if Phase 3 shows unresolved multi-hop questions that cannot be handled adequately by a fixed hybrid pipeline.

A future agent may:

- Inspect evidence sufficiency using a validated typed decision.
- Decompose/rephrase a question or perform one bounded additional retrieval.
- Combine evidence from the same approved, scoped corpus.
- Abstain when supporting evidence remains insufficient.

Initial proposed constraints: at most two retrieval rounds and one query rewrite, fixed deadline/token/cost caps, no arbitrary URLs or SQL, no write tools, and no automatic connector ingestion. Every round must preserve authenticated scope, record its evidence, and use the same citation validation.

This remains an independent opt-in mode with the deterministic pipeline as fallback. A future agent framework is not selected or installed by this design.

9. Evaluation and acceptance gates

The existing 76-test pass and TypeScript check are baseline observations from the analysis, not evidence that this proposal works. No implementation tests were added or executed for this documentation-only change.

Use a versioned, non-sensitive benchmark covering exact identifiers, semantic paraphrases, procedures, numerical facts, multi-document questions, unanswerable requests, multilingual queries, and conflicting evidence. Include metadata filters and realistic document sizes.

Proposed acceptance gates, to approve before implementation:

Gate	Required evidence
Default compatibility	Existing deterministic tests pass; fixed valid dense-mode requests keep their expected outputs. Intentional bug fixes are documented separately.
Isolation	Zero unauthorized/cross-filter/cross-store candidates or cache reuse in the complete regression matrix. Test all seven scope/metadata fields and missing/default values.
Backend fidelity	Genuine IDs/metadata/distances, unique IDs after deletion, and consistent scoped add/clear/search for FAISS and real PostgreSQL.
Retrieval quality	Macro Recall@5 at least baseline; on the exact-term subset, initially target at least 5 percentage points higher Recall@5. Reject material semantic-subset regressions; thresholds need corpus review.
Evidence quality	Every emitted structured citation maps to authorized retained evidence; no invented source IDs/pages. Claim support still requires separate validation/evaluation.
Latency	Initially target warm retrieval p95 no more than 1.25x dense baseline on the same machine/corpus/concurrency. Absolute SLOs require measured workload agreement.
Resource control	Candidate pools, prompt size, downloads, queue backlog, and optional retry/agent budgets have tested bounds.

Gate	Required evidence
Rollback	Flag-off restores the dense route without data deletion or cache-mode confusion.

The recall/latency values are **proposed targets, not measured results or guarantees**. If quality does not improve or the latency budget is exceeded, keep the feature off and investigate rather than declaring success.

Tests should cover:

- Keyword evidence absent from the dense candidate list.
- RRF duplicates, 1-based ranks, ties, missing-branch membership, and pool caps.
- Same text in two sources with distinct identities.
- Successful empty branches versus backend failure.
- Scope predicates on lexical and dense queries.
- Cache identity across every filter, mode, query, store, and generation.
- Add/delete/replacement concurrency and failed update recovery.
- Monolith versus split retrieval parity and configuration mismatch.
- Real PostgreSQL index/query behavior and generation invalidation.
- Unchanged API/client response compatibility and async job completion.

No numeric coverage target should substitute for these behavioral tests.

10. Expected future change surfaces

These are implementation targets, not edits made in this turn.

Location	Planned change
`app\config.py`	Strict opt-in flag parsing and backend/topology validation.
`app\vector_store.py`	Genuine common candidate contract; backend lexical search and update consistency; preserve dense search interface.
`app\rag.py`	Mode dispatch, complete cache keys, structured evidence retention, fusion integration, later citation validation.
`app\retrieval_service.py`	Shared retrieval helper and additive typed internal contract; mode/capability checks.
`app\main.py`	Prerequisite scoped/atomic indexing and safe staging; later authentication/readiness/metrics. Public paths preserved.
`app\schemas.py`	Later additive structured sources/degraded status after contract review.
`app\ingestion.py`, `db_ingestion.py`, `chunking.py`	Later provenance preservation and bounded extraction/chunking improvements.
`app\streamlit_demo.py`	Reuse correct scoped ingestion/retrieval paths instead of unscoped demo storage.

Location	Planned change
`frontend\RAGQAApp.tsx`, `frontend\src\services.ts`, `types.ts`	Correct async/API contracts without changing layout. No UI changes required for initial fusion.
`scripts\create_pgvector_table.sql` and a reviewed optional migration	Add lexical/filter indexes and shared generations only when approved.
`tests\`	Extend existing modules; add focused lexical/fusion/service/integration tests where necessary.
`requirements.txt`	Change only if a chosen, reviewed lexical/reranking dependency is actually required.
`README.md`, deployment examples, `Jenkinsfile`	Document feature defaults, schema readiness, supported topology, rollback, and verified checks.

11. Risks and explicit non-goals

Hybrid retrieval cannot automatically repair missing parsed content, fabricated answers, poor chunk boundaries, authorization gaps, stale caches, or ambiguous source IDs. Those are separate constraints.

BM25/FTS configurations can favor different query types. RRF can combine weak results as well as strong ones. Lexical indexes consume resources; selective metadata queries require real query-plan/recall tests. A learned reranker adds model latency/dependencies and is not presumed necessary.

No proposal to replace FastAPI, React, SentenceTransformers, FAISS/pgvector, or the existing provider integrations is made. No knowledge graph, autonomous web/SQL agent, multi-agent fleet, fine-tuning, local generative model, or new managed search service is included in the initial design.

12. Decision boundary

This document is ready for review. The next implementation approval should specify a small first batch, not the entire roadmap at once.

Recommended first batch: Phase 0 cache/identity/result-contract fixes with regression tests, followed by Phase 1 opt-in lexical+dense fusion. Wider exposure and multi-worker use remain blocked until their respective security/consistency gates pass.

The current application continues to run with its existing configuration. Nothing was enabled, migrated, restarted, committed, or pushed for this proposal.